Projet de Programmation Système Avancée : HACHEMENT

(Hachage Avec Chaînage Hébergé En Mémoire Et Non Transitoire)

M1 Informatique 2026

1 Description du projet

Le but du projet est d'implémenter une table de hachage persistante capable de stocker divers objets. La persistance veut dire que la table existe dans un fichier régulier et survit à la fin des processus la manipulant. Mais c'est bien une table, donc une structure mémoire : on ne pourra y accéder que via une projection avec `mmap`. J'insiste : tout accès via les primitives d'entrée/sortie comme `read` ou `write` `fread` etc. est **interdit**.

La table contient b **blocs de 1ko**. Il ne faut utiliser aucune autre structure de données que les blocs : les données persistantes manipulées par votre code doivent être un fichier de b kilo-octets, rien d'autre.

2 Fonction de hachage

La fonction de hachage, notée *h(C)*, prend un objet binaire C de n octets et retourne un nombre non signé, nommé le `hashcode` de C, compris entre 0 (inclus) et b (exclu). Vous devez implémenter au moins les deux fonctions de hachage suivantes (et éventuellement d'autres, à décrire dans le rapport).

Pour ne pas surcharger la ligne de commande, c'est le **nom de fichier** qui permettra de savoir le type de fonction de hachage utilisé :

— Un nom en `*.cks` pour usage de la fonction Checksum32$_b$

— Un nom en `*.sha` pour usage de la fonction Sha-1$_b$

Notez que la valeur de b se récupère grâce à `fstat`.

2.1 Checksum32$_b$

On considère que C est une suite de mots de 4 octets, c'est-à-dire de $n/4$ `uint32_t` $c_1 ... c_{n/4}$.

$$Checksum32_b(C) = \left(\sum_i c_i \right) \mod b$$

Dans la formule précédente, la somme est celle des `uint32_t` c'est-à-dire que toutes les sommes de deux termes sont faites modulo 2^{32}, avant un dernier modulo b final.

Dans le cas où n n'est pas multiple de 4, on ajoute de un à trois octets de bourrage égaux à 0 à la fin de C. Par exemple pour $C = 1$ (C n'a qu'un seul octet égal à 1), Checksum32$_b(C)$ est 2^{24} mod b et non pas 1 mod b.

Notez que l'on ne prévoit pas d'utiliser $b > 2^{32}$, ce qui laisse toutefois de la marge : on peut faire une table de $4To$ maximum.

L'intérêt de la fonction Checksum32$_b$ est qu'il est facile de générer des **collisions** : étant donné C et un hashcode cible x, on fabrique C' tel que Checksum32$_b(C') = x$ en commençant par étendre C en C'' avec d'éventuels octets de bourrage, puis en rajoutant en fin de C'' quatre octets valant $(x - Checksum32_b(C'')) \bmod b$.

2.2 Sha1$_b$

Une fonction plus sérieuse de hachage est `SHA-1`. On trouvera sa description par exemple sur `https://fr.wikipedia.org/wiki/SHA-1`.

$$Sha1_b(C) = (SHA1(C) \bmod 2^{32}) \bmod b$$

$SHA1(C)$ est un nombre sur 20 octets ; on prend donc les 4 derniers (puisque $b < 2^{32}$) avant d'y appliquer un modulo b. Un code est fourni sur Moodle pour calculer le SHA-1 de C d'un fichier projeté en mémoire. Il utilise la librairie OpenSSL `libcrypto` et est donc à compiler avec `gcc mmap_sha1.c -lcrypto`.

3 Structure d'un bloc et gestion des collisions

Le principe de base de la table est qu'un objet C sera stocké dans le bloc numéro $h(C)$. Cependant, tous les blocs ont la même taille de 1ko, alors que les objets ont une taille quelconque. De plus, des **collisions** peuvent se produire, c'est-à-dire que plusieurs objets peuvent avoir le même hash (cela devient de plus en plus probable à mesure que la table se remplit, même en utilisant une fonction de hachage universelle).

On gérera ces deux problèmes en utilisant le **chaînage interne**. Chaque bloc pointe un bloc **suivant**, ce qui définit une liste chaînée de blocs. Cette liste chaînée contient les blocs constituant l'objet C (par exemple si C fait 3500 octets il faut une liste chaînée de 4 blocs). On doit donc stocker la taille n de C.

Quand on arrive à la fin d'une entrée (dans notre exemple, au quatrième bloc) il faut savoir si on termine là, ou si l'on continue pour l'entrée suivante C' qui collisionne avec C, c'est-à-dire que $h(C) = h(C') = a$ et on a commencé la recherche au bloc numéro a.

Que faire si l'objet C utilise donc quatre blocs de numéro $a = h(C)$, c, d et e, et qu'on doit insérer un autre objet C'' avec $h(C'') = c$? La solution retenue ici est de pointer un autre bloc nommé **premier**. Quand on a calculé $h(C'') = c$, on regarde si $premier(c) == c$. Si oui, ce bloc est le début de la liste chainée des objets de hashcode c. Si non, cette liste commence en $premier(c)$. D'une façon gérérale, pour un hashcode h, la tête de la liste des objets de hashcode h se trouve en $premier(h)$ (ou en `Table[h].premier` si vous préférez).

3.1 Calcul du premier et du prochain bloc

Quelle valeur donner à `premier` (si le bloc cible est déjà utilisé) et à `suivant`, en cas de collision ou de stockage d'un objet d'une taille débordant un bloc ? On peut utiliser plusieurs techniques : le *sondage linéaire*, c'est-à-dire le prochain bloc libre ; le *sondage quadratique*, le *double hachage*... Vous pouvez regarder `https://fr.wikipedia.org/wiki/Table_de_hachage#Adressage_ouvert` Préciser dans le rapport la méthode utilisée.

3.2 Format d'une entrée

La table est constituée de b blocs de même taille : 1024 octets. Chaque bloc doit avoir la même structure, définie dans votre rapport. Elle doit être une variation de

```
struct hache_entree {
    uint32_t premier; // numero du premier bloc de la liste chainée
    uint32_t suivant; // numero du bloc suivant de la liste chainée
    uint32_t longueur; // longueur, en octets, de l'objet stocké.
    char obj[1012];  // l'objet stocké
};
```

Variation voulant dire qu'on peut y ajouter des champs, comme un `type` en particulier, en diminuant la taille de `obj`. On utilise les valeurs spéciales

- `suivant=0xffffffffUL` pour signifier que le bloc est le dernier du chaînage
- `longueur=0xffffffffUL` pour signifier que le bloc est libre (ne contient aucune donnée)

Et on rappelle que dans une table vierge, chaque bloc à l'indice i doit avoir son champ `premier` valant i.

Notez que la taille maximum d'un objet stocké est limitée par `longueur` à $2^{32} - 2$ octets. Ce champ `longueur` sert aussi à savoir si un nouvel objet commence ou pas : **si** la longueur est inférieure ou égale à ce que l'on peut stocker (ici $1012 = 1024 - 3 * 4$, mais si vous rajoutez d'autres champs cette valeur sera plus petite) **et si** `suivant` n'est pas le marqueur `0xffffffffUL` de fin de chaîne, **alors** le bloc suivant contient un nouvel objet. Ainsi, dans l'exemple où l'objet C fait 3500 octet, le premier bloc aura comme `longueur` 3500, le deuxième 2488, le troisième 1476, et le quatrième 472, ce qui indique qu'un nouvel objet commence au cinquième bloc de la liste chaînée.

4 Objets dans la table

On doit pouvoir insérer des objets d'au moins ces trois types. Vous pouvez, en extension, proposer d'autre types, mais cela doit avoir un intérêt (posez sur le forum la question de la pertinence).

4.1 Chaîne de caractères

L'objet stocké est la chaîne elle-même. Cela permet de recalculer le hashcode, pour faire un test d'intégrité et vérifier s'il y a eu de la corruption (cf 5.8).

4.2 Fichier

Quand on insère un fichier dans la table, on doit y mettre son contenu ainsi que ses métadonnées, telles que nom, taille, date de modification... Il faut prévoir un format adéquat pour cela. Les métadonnées seront hachées, après avoir été concaténées avec les données du fichier, et donc une modification d'une métadonnée entraîne un hashcode différent et donc est une entrée différente. Ainsi il faut éviter de mettre la date de dernier accès (`atime`) dans les métadonnées. **Précisiez dans le rapport** comment vous stockez un fichier, et quelles métadonnées, suivant quel format.

4.3 Nom de fichier

Un nom de fichier est une chaîne de caractères plus un mécanisme permettant de désigner le, ou les, objets fichiers stockés dont le nom correspond à cette chaîne. Cela permet de faire de la recherche par nom avec la commande `lookup`, cf section suivante. Précisez dans votre rapport comment vous implémentez cela.

En effet, vous devez prévoir une fonction de **recherche d'un fichier par nom**. Ainsi la recherche de `toto.txt` doit donner **tous** les fichiers qui ont été insérés dans la table sous le nom `toto.txt`. Une façon efficace d'implémenter (plus efficace que parcourir toute la table !) est d'utiliser ce type d'entrée *nom de fichier*, dont le contenu pointe les hashcodes correspondants au fichier.

Au passage cela permet aussi d'informer, quand on insère un fichier dans la base et qu'un fichier de contenu ou de métadonnées différents s'y trouve déjà avec le même nom.

5 Commandes à implémenter

Toutes les commandes pour accéder aux fichiers prendront la forme
`hachement commande table slowns [autres paramètres]`
où `table` désigne toujours le nom du fichier contenant la table de hachage et `slowns` un nombre de nanosecondes durant lequel attendre, cf section 6. On peut rajouter des extensions à la ligne de commande (options préfixée d'un tiret comme `-v` etc.) **à préciser dans le rapport !** Les commandes à implémenter sont :

5.1 `make`

`hachement make table.sha 0 b` crée une nouvelle table nommée `table.sha` de b blocs de 1ko (tous marqués comme libres). Dans cet exemple, l'extension en `*.sha` indique que c'est pour usage avec la fonction de hachage SHA-1p.

5.2 `free`

Affiche la quantité de blocs libres et occupés, ainsi que le taux de remplissage. Si disponibles on pourra aussi afficher d'autres informations : nombre de chaînes, nombre de fichiers, nombre total d'objets dans la table...

5.3 `insertstring`

Insère dans la table une chaîne de caractères, lue en ligne de commande :
`hachement insertstring table.sha 0 Hello, world!`
par exemple pour insérer la chaine *"Hello, world !"* (une seule chaîne). Si la chaîne est déjà présente, un *warning* doit être affiché et la table n'est pas modifiée.

5.4 `insertfile`

Insère un fichier (dont le nom est donné en paramètre) dans la table. Si le fichier est déjà présent, au sens même contenu et même métadonnées, un *warning* doit être affiché et la table n'est pas modifiée.

5.5 `lookup`

Recherche si la chaîne passée en ligne de commande a été insérée dans la table ou (inclusif) si *des* fichiers portant ce nom existent. Selon le cas on pourra demander (interactivement ou selon une option de la ligne de commande) quoi faire des fichiers (sauvegarder, afficher ou juste notifier)

5.6 `deletehash`

Supprimer la (ou toutes les) entrée(s) dont le hascode est donné (sous forme décimale ou hexa, à préciser). Un *warning* sera affiché en cas d'entrée vide. Notez qu'au niveau algorithmique ce n'est pas trivial : vous pouvez choisir de faire une supression en temps constant juste en modifiant les pointeurs `premier` et `suivant` mais il est plus intéressant dans certains cas de déplacer des blocs. Pas trop non plus ! À préciser dans le rapport.

5.7 `delete`

Supprimer une ou plusieurs entrées : chaîne ou (inclusifs) fichiers dont le nom correspond

5.8 `check`

Vérifie la correction de la table de hachage. En particulier, que pour tout bloc occupé, le contenu correspond au hashcode attendu. Une chose intéressante est en effet de lutter contre la **corruption** [1] : que se passe-t-il si des bits sont changés aléatoirement dans la table ? Il y aura incohérence entre le hashcode recalculé et l'adresse où est stockée la donnée (correspondant au hashcode initial). On devrait pouvoir le détecter. Un code `corrupt.c` est disponible sur Moodle pour tester cela.

5.9 `extend/shrink/ajust`

Le but de ces trois commandes est de redimensionner la table, c'est-à-dire de changer son nombre b de blocs en gardant le même contenu. `extend` **double**

1. n'y voyez aucune allusion politique

la taille de la table tandis que `shrink` la **divise par deux**, et `ajust` appelle l'une de ces deux commandes si besoin, en utilisant la règle suivante :

— si le taux de remplissage dépasse 70%, alors `extend`, sinon
— si ce taux est inférieur à 10%, alors `shrink`,
— sinon, rien.

`extend` (et éventuellement `shrink`) demande à recalculer les hashcodes de toutes les entrées et à faire un nouveau fichier, en écrasant au final l'ancien contenu. Il est intéressant d'implémenter cela en **multithread**. Les commandes ne prennent aucun paramètre supplémentaire ; par exemple :

```
hachement extend table.chk 0
```

6 Accès concurrents et verrouillage

Un aspect important du projet est la concurrence, c'est-à-dire le fait que plusieurs processus puissent tenter d'intervenir ensemble sur la table de hachage sans créer d'incohérence. Il est **nécéssaire** d'implémenter un verouillage POSIX (`fcntl`), au niveau des blocs de 1ko (pas global, sauf pour `make`) de façon à gérer les *race conditions* d'accès à un bloc. De plus, chaque accès à un bloc de la table **doit** respecter le schéma suivant :

1. Obtenir un verrou (en lecture ou écriture, selon le cas) sur le bloc
2. Réaliser l'opération demandée
3. **Attendre** `slowns` nanosecondes, grâce à l'appel système `nanosleep`, si le paramètre `slowns` en ligne de commande n'est pas 0
4. Relâcher le verrou, ou le cas échéant le convertir et aller en 1

Cette attente simule la lenteur du disque dur, et permet de tester la concurrence : des *race conditions* d'accès lents aux mêmes blocs doivent être évitées.

7 Insertion multithread de toutes les lignes d'un fichier

Une fonctionalité utile et permettant de tester votre maîtrise de la programmation multi-threads est la suivante. Il s'agit de prendre un gros fichier texte (comme par exemple `dictionnaire_officiel_scrabble_2012.txt` fourni sur Moodle), de le projeter en mémoire (rappel, on n'a pas le droit d'utiliser `fgets` ou autre fonction d'entrée/sortie séquentielle comme `fgetc`, `fscanf`...) et d'insérer chacune de ses lignes dans la table, comme une chaîne de caractères.

Il est **obligatoire** d'utiliser plusieurs **threads** pour cela. Une stratégie serait d'en mettre un par ligne. Mais comment repérer les fins lignes (`'\n'`) sans créer un goulot d'étranglement séquentiel ? Une meilleure stratégie est de couper le fichier en zones de même taille (par exemple, 5% du total, ou 10ko. Idéalement, ça doit être paramètrable) et d'affecter un thread par zone. Attention à insérer une et une seule fois les lignes franchissant les frontières de zones.

8 Modalités d'évaluation

8.1 Inscription

Le projet est à faire en équipe de **deux personnes** (binôme) mais peut être fait seul sur dérogation écrite obtenue avant le 17 mars. Chaque personne faisant le projet doit envoyer par mail à `fm@irif.fr` au plus tard le **17 mars 2026**, la composition de votre équipe (nom et numéro étudiant des deux membres) en mettant l'autre membre de l'équipe en copie du mail.

8.2 git

Chaque équipe doit créer un dépôt `git` **privé** sur le gitlab de l'UFR `https://moule.informatique.univ-paris-diderot.fr/` et y donner accès en tant que *Reporter* à Fabien de Montgolfier (`demontgo`) et Anne Micheli (`micheli`). Le dépôt devra contenir un rapport **au format pdf** donnant la liste des membres de l'équipe (avec numéro étudiant et login sur le gitlab) ainsi qu'un **Makefile** tel que l'exécution de `make` à la racine du dépôt crée (dans ce même répertoire) l'exécutable `hachement`.

8.3 Attendus et calcul de la note

Ce qui est évalué est le respect des spécifications : plus on a implémenté correctement de choses, meilleure est la note ; la robustesse du code (détection d'erreurs internes et externes) ; et la vitesse d'insertion de toutes les lignes d'un fichier ou de plusieurs gros fichier (faire des *benchmarks* sur lulu).

Pour rappel, l'emprunt de code sans citer sa source est un plagiat. L'emprunt de code en citant sa source est autorisé, mais bien sûr vous n'êtes notés que sur ce que vous avez effectivement produit. Le plagiat **même partiel** entraîne une note de 0 et une éventuelle convocation de la section disciplinaire.

On prendra grand soin au **traitement des erreurs**, autant les erreurs système (l'appel système retourne -1 etc.) que les erreurs internes à l'application, telles que la table est pleine ou de la corruption est détectée (cf 5.8) etc. Un message explicite sera affiché sur l'erreur standard. En cas de succès les commandes sont muettes, sauf explicitement demandé comme `free` : évitez la verbosité inutile.

On peut distiguer globalement six paliers dans la réalisation du projet

1. Insérer, rechercher et supprimer des chaînes dans la table, sans gestion des collisions
2. idem, avec gestion des collisions et des chaînes de plus de 1ko
3. gestion des fichiers
4. Bonne gestion du verrouillage avec plusieurs processus ralentis par `slowns` insérant concuremment de gros fichiers
5. Insertion de toutes les lignes d'un fichier et redimentionnement de la table, avec une bonne implémentation du **multi-thread**. Soignez cela car cela permet d'évaluer votre bonne compréhension de la concurrence, c'est la partie du projet qui permet d'avoir une bonne note.
6. Benchmarking et extensions (non triviales : l'ajout de fonctionalités pas intéressantes n'augmentera pas la note. Dans le doute, demandez !)

8.4 Attendus examinés lors de la soutenance

1. Les deux membres de l'équipe peuvent expliquer le code
2. Le projet pourra être présenté et testé sur votre machine mais doit s'exécuter correctement sur lulu.
3. Les commandes font ce qu'il faut
4. Des scénarios faisant intervenir la concurrence sont testés

Si un membre de l'équipe a moins d'activité `git` et répond moins bien aux questions, il sera moins bien noté, jusqu'à 0 en cas d'absence de travail, même si le projet est bon.

8.5 Portabilité

Le sujet doit compiler sous Linux avec `gcc` sans bibliothèques exotiques. Les bibliothèques standard, POSIX threads `pthread` et OpenSSL `crypto` sont autorisées, pour d'autres bibliothèques, posez la question, la valeur par défaut est *non autorisé*. Vous avez le droit d'utiliser des appels système non-POSIX seulement s'ils sont Linux. Le sujet fait comme hypothèse que la machine est *little endian*[2] (on vérifie que `htonl(42) != 42`). Sinon, il faudra présenter sur les machines de l'UFR pour que les tests puissent bien s'effectuer.

2. à ceux qui diraient que ce n'est pas portable, on peut répondre que le format GIF89a, sans lequel le Web ne serait pas le Web, est *little endian*